

Lecture 02b:

Larger Than Memory Files



Markus Hohle

University of California, Berkeley

Data Science for Scientific
Computing

MSSE 277A, 3 Units



Outline

Standard Pandas

Line By Line

Chunks With Dask

PySpark



Outline

Standard Pandas

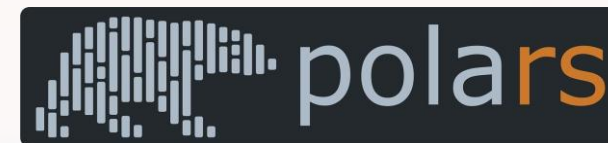
Line By Line

Chunks With Dask

PySpark



Picking the right tool matters!



.xlsx

83 sec

na

8.2 sec
10.2 sec

.csv

1.2 sec

0.016 sec
1.6 sec

0.27 sec
0.20 sec

.txt

1.5 sec

0.016 sec
0.92 sec

0.25 sec
0.23 sec

check out `Benchmark_Pandas_Dask_Polars.py`



sometimes, a file is larger than the machines memory

- exploring the file **line wise**
- reading only that **part** of the data which is **actually needed**
- reading/writing in **chunks**

Let us work with a dataset that is actually *smaller* than the memory

idea: comparing different methods (including pandas) and measure **run time** and **memory usage**

```
import time
```

decorator
to measure runtime

```
def my_timer(my_function):
```

```
    def get_args(*args,**kwargs):
```

```
        t1 = time.monotonic()
```

```
        results = my_function(*args,**kwargs)
```

```
        t2 = time.monotonic()
```

```
        dt = t2 - t1
```

```
        print("Total runtime: " + str(dt) + " seconds")
```

```
        return results, dt
```

```
    return get_args
```



idea: comparing different methods (including pandas) and measure **run time** and **memory usage**

```
import psutil
import functools
import tracemalloc
```

*decorator
to measure memory usage*

```
def memory_usage_decorator(func):
    @functools.wraps(func)
    def wrapper(*args, **kwargs):
        tracemalloc.start()
        result = func(*args, **kwargs)
        current, peak = tracemalloc.get_traced_memory()
        tracemalloc.stop()
        print(f"Memory usage for {func.__name__}:")
        print(f"  Current: {current / (1024 * 1024):.2f} MB")
        print(f"  Peak:    {peak / (1024 * 1024):.2f} MB")
        return result
    return wrapper
```

memory currently used (varies
over time)

peak memory (maximum), most
relevant for reading/writing process

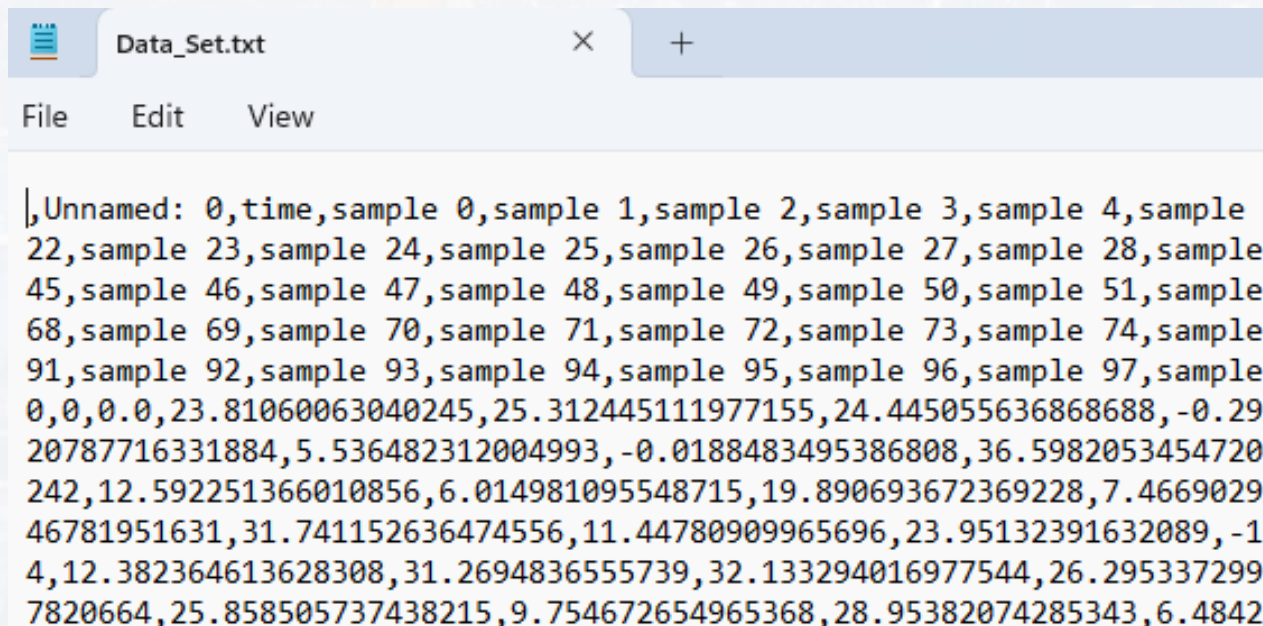


@my_timer

@memory_usage_decorator

```
def StandardPandas(file_path: str = '../Datasets/Data_Set.txt'):
    return pd.read_csv(file_path)
```

Data Set.txt ≈ 182 MB





idea: comparing different methods (including pandas) and measure **run time** and **memory usage**

@my_timer

@memory_usage_decorator

```
def StandardPandas(file_path: str = '../Datasets/Data_Set.txt'):
    return pd.read_csv(file_path)
```

Data_Set.txt ≈ 182 MB

StandardPandas()

Memory usage for StandardPandas:

Current: 78.63 MB
Peak: 157.26 MB

Total runtime: 1.2190000000118744 seconds

Out[12]:

	Unnamed: 0.1	Unnamed: 0	time	...	sample 97
0	0	0	0.000	...	18.899622
1	1	1	0.001	...	16.669575
2	2	2	0.002	...	15.750601
3	3	3	0.003	...	16.527525
4	4	4	0.004	...	18.793216
...	...	...	...	...	...
99995	99995	99995	99.995	...	19.910798
99996	99996	99996	99.996	...	22.044467
99997	99997	99997	99.997	...	25.177334
99998	99998	99998	99.998	...	28.300678
99999	99999	99999	99.999	...	30.434906

[100000 rows x 103 columns],

(1.2190000000118744)

peak memory corresponds
approximately to file size

well structured file

total runtime in seconds



Outline

Standard Pandas

Line By Line

Chunks With Dask

PySpark



often, we don't know the structure of a data file.

if too large: **read first entries line by line** and print/store the output

```
@my_timer
```

```
@memory_usage_decorator
```

```
def TextFileLineByLine1(file_path: str = '../Datasets/Data_Set.txt',\n                        k: int = 5):
```

```
    with open(file_path, 'r') as file:\n        for i, line in enumerate(file):\n            if i >= k:\n                break\n            print(line)
```

reading the first five lines:

```
TextFileLineByLine1()
```

```
Memory usage for TextFileLineByLine1:
```

```
Current: 0.15 MB
```

```
Peak: 0.18 MB
```

```
Total runtime: 0.01600000000325963 seconds
```

minimal memory consumption



reading the first five lines:
TextFileLineByLine1()

column names (keywords),
separated by comma

data is structured

```
In [13]: TextFileLineByLine1()  
,Unnamed: 0,time,sample 0,sample 1,samp  
12,sample 13,sample 14,sample 15,sample  
26,sample 27,sample 28,sample 29,sample  
40,sample 41,sample 42,sample 43,sample  
54,sample 55,sample 56,sample 57,sample  
68,sample 69,sample 70,sample 71,sample  
82,sample 83,sample 84,sample 85,sample  
96,sample 97,sample 98,sample 99
```

```
0,  
0,0.0,23.81060063040245,25.3124451119771  
2740792067254,5.155855356386692,15.70238  
8,36.59820534547204,16.936315893981185,1  
437353255,18.05148342734458,6.8558275083  
42263420235605,-1.6129158279890152,21.90  
1631,31.741152636474556,11.4478090996569  
446949008987,13.279841996028694,35.92034  
,26.29533729976512,23.282536825428583,17  
7308402,26.54782218211423,31.25618889782  
352444483523,26.95935326436113,14.048154  
.509918246417644,12.424489553318868,16.5  
02,6.71431687590743,19.439262989661604,3  
6513,3.7759978478618814,9.6595054971236,  
647936
```

```
1,  
1,0.001,23.24191960143792,24.75576430971  
6790093636127,4.718116935111938,15.30209  
5.14989069912229,16.938697410251578,10.2  
59,18.63793433138781,6.903883334547536,1  
687214,-0.3577945473491559,21.9051618406  
3706376091183,13.340358676360577,23.0767
```




often we need only parts of the data (say columns 1, 3, 5 etc) → keyword search to **find columns!**

```
@my_timer
```

```
@memory_usage_decorator
```

```
def TextFileLineByLine2(file_path: str = '../Datasets/Data_Set.txt', **keywords):
```

```
    locations = {v: None for v in keywords.values()}
```

pre-allocating empty **dict**
for storing the column
which belongs to the
keyword

```
    with open(file_path, 'r') as file:
```

```
        for line in file:
```

```
            ListSplit = line.split(',')
```

we saw earlier that column
names are separated by
comma (usually **not** hardcoded)

```
            for k in locations.keys():
```

```
                if locations[k] is None and k in ListSplit:
```

```
                    locations[k] = ListSplit.index(k)
```

```
            if all(locations.values()):  
                break
```

we are done, once all
columns that we need are
located



often we need only parts of the data (say columns 1, 3, 5 etc) → keyword search to **find columns!**

@my_timer

@memory_usage_decorator

```
def TextFileLineByLine2(file_path: str = '../Datasets/Data_Set.txt', **keywords):
```

```
    locations = {v: None for v in keywords.values()}
```

```
    ...
```

```
        if all(locations.values()):  
            break
```

```
        if not all(locations.values()):  
            print('the following keywords were not found:\n')  
            for k, v in zip(locations.keys(), locations.values()):  
                if not v:  
                    print(k + '\n')
```

```
    return locations
```

in case some keywords
were not found



often we need only parts of the data (say columns 1, 3, 5 etc) → keyword search to **find columns!**

```
(Location, _) = TextFileLineByLine2(word1 = 'time', word2 = 'sample 0', \
                                     word3 = 'sample 4')
```

output form decorator
not needed

Memory usage for TextFileLineByLine2:

Current: 0.00 MB
Peak: 0.03 MB

Total runtime: 0.0 seconds

minimal memory consumption

Location
{'time': 2, 'sample 0': 3, 'sample 4': 7}

data of time, sample 0 and
sample 4 are located in columns
2, 3, and 7 respectively



often we need only parts of the data (say columns 1, 3, 5 etc) → keyword search to **find columns!**

```
(Location, _) = TextFileLineByLine2(word1 = 'time', word2 = 'sample 0', \
                                     word3 = 'XYZ', word4 = 'Sample 0')
```

```
the following keywords were not found:
```

```
XYZ
```

```
Sample 0
```

```
Memory usage for TextFileLineByLine2:
Current: 0.00 MB
Peak:    0.04 MB
Total runtime: 7.047000000020489 seconds
```

```
Location
{'time': 2, 'sample 0': 3, 'XYZ': None, 'Sample 0': None}
```

upper case!

Doesn't find keyword, if not consistent. **Requires fuzzy match** (see next lecture!)

minimal memory consumption, but needs to search the entire file, which takes more time



often we need only parts of the data (say columns 1, 3, 5 etc) → keyword search to **find columns!**

```
(Location, _) = TextFileLineByLine3(word1 = 'time', word2 = 'sample 0', \
                                     word3 = 'XYZ', word4 = 'Sample 0')
```

Adding lines for reading specific columns once found in the data:

```
L = [[] for i in range(len(keywords))]
```

```
with open(file_path, 'r') as file:
    for l, line in enumerate(file):
```

```
        if l>1
```

```
            ListSplit = line.split(',')
            for i, (k, v) in enumerate(zip(locations.keys(),
```

```
                                       locations.values())):
```

```
                L[i].append(ListSplit[v])
```

preparing empty list

selecting the
columns

saving content as list



With some small modifications:

storing the data once we located it in the file

```
A      = np.array(L, dtype = float).T  
A_df    = pd.DataFrame(A, columns = locations.keys())
```

turning extracted data
into dataframe

```
A_df.to_csv('ExtractedData.csv', index = False)
```

saving data as .csv

```
with open('ExtractedData.pkl', 'wb') as f:  
    pickle.dump(A_df, f)
```

storing as .pkl file

.pkl files

- needs `import pickle`
- not human readable
- requires **half the space** of a .csv file
- reading is faster
- preferred format, if large data set has to be stored and **read during workflow**

```
with open(filename, 'rb') as file:  
    loaded_data = pickle.load(file)
```




With some small modifications:

storing the data once we located it in the file

```
A      = np.array(L, dtype = float).T  
A_df   = pd.DataFrame(A, columns = locations.keys())
```

turning extracted data
into dataframe

```
A_df.to_csv('ExtractedData.csv', index = False)
```

saving data as .csv

```
with open('ExtractedData.pkl', 'wb') as f:  
    pickle.dump(A_df, f)
```

storing as .pkl file

Microsoft Excel Comma Separated Values File



ExtractedData

19/10/2025 01:47

Microsoft Excel Com...

4.542 KB

PKL File



ExtractedData.pkl

19/10/2025 01:47

PKL File

2.345 KB



With some small modifications:

storing the data once we located it in the file

```
A      = np.array(L, dtype = float).T  
A_df   = pd.DataFrame(A, columns = locations.keys())
```

turning extracted data
into dataframe

```
A_df.to_csv('ExtractedData.csv', index = False)
```

saving data as .csv

```
with open('ExtractedData.pkl', 'wb') as f:  
    pickle.dump(A_df, f)
```

storing as .pkl file

```
print(A_df.head())
```

	time	sample 0	sample 4
0	0.001	23.241920	7.778409
1	0.002	22.002649	8.569085
2	0.003	20.490190	9.503710
3	0.004	19.192838	10.498987
4	0.005	18.527045	11.466261

Memory usage for TextFileLineByLine3:

Current: 51.78 MB

Peak: 91.39 MB

Total runtime: 25.984999999998603 seconds



Outline

Standard Pandas

Line By Line

Chunks With Dask

PySpark



dask uses **parallel processing**



idea: reading and writing data in **chunks** in order to save memory!

.xlsx

83 sec

na

.csv

1.2 sec

0.016 sec

1.6 sec

.txt

1.5 sec

0.016 sec

0.92 sec



```
@memory_usage_decorator  
def Chunks(df):  
    return [chunk.compute() for chunk in df.to_delayed()]
```

```
@my_timer  
def CSVFileDask1(file_path: str = '../Datasets/Data_Set.csv',\  
                 blocksize: int = 2):
```

```
    df = dd.read_csv(file_path, blocksize = int(blocksize * 1024 * 1024))
```

```
    with ProgressBar():  
        chunks = Chunks(df)
```

```
    return pd.concat(chunks, ignore_index = True)
```

reading the file in
chunks of defined size
(=blocksize in MB)

compute turns dask
objects into actual
dataframes



```
@memory_usage_decorator
```

```
def Chunks(df):
```

```
    return [chunk.compute() for chunk in df.to_delayed()]
```

```
@my_timer
```

```
def CSVFileDask1(file_path: str = '../Datasets/Data_Set.csv',\n                 blocksize: int = 2):
```

```
    df = dd.read_csv(file_path, blocksize = int(blocksize * 1024 * 1024))
```

```
    with ProgressBar():\n        chunks = Chunks(df)
```

```
    return pd.concat(chunks, ignore_index = True)
```

```
out = CSVFileDask1(blocksize = 200)
```

```
[#####] | 100% Completed | 1.47 ss
```

```
Memory usage for Chunks:
```

```
    Current: 77.89 MB
```

```
    Peak:    512.74 MB
```

```
Total runtime: 1.5 seconds
```

fast, but memory intense!



```
@memory_usage_decorator
```

```
def Chunks(df):
```

```
    return [chunk.compute() for chunk in df.to_delayed()]
```

```
@my_timer
```

```
def CSVFileDask1(file_path: str = '../Datasets/Data_Set.csv',\n                 blocksize: int = 2):
```

```
    df = dd.read_csv(file_path, blocksize = int(blocksize * 1024 * 1024))
```

```
    with ProgressBar():  
        chunks = Chunks(df)
```

```
    return pd.concat(chunks, ignore_index = True)
```

```
out = CSVFileDask1(blocksize = 2)
```

slower, but requires less
memory

Memory usage for Chunks:

Current: 80.02 MB

Peak: 85.15 MB

Total runtime: 9.984000000025844 seconds



Memory usage for Chunks:

Current: 80.02 MB

Peak: 85.15 MB

Total runtime: 9.984000000025844 seconds

needs way more memory
than blocksize!
reason: file is still stored
in memory!

solution: **saving data in chunks as we read it!**



solution: **saving data in chunks as we read it!**

`@memory_usage_decorator`

```
def ChunksSave(df, outfilename: str):
```

```
    for i, chunk in enumerate(df.to_delayed()):  
        computed_part = chunk.compute()
```

```
        if i == 0:  
            computed_part.to_csv(outfilename, mode = 'w', index = False,\  
                                header = True)  
        else:  
            computed_part.to_csv(outfilename, mode = 'a', index = False,\  
                                header = False)
```

compute turns dask
objects into actual
dataframes

actual saving part:
flag 'w' stands for *write*
flag 'a' stands for *append*



@my_timer

```
def CSVFileDask2(file_path: str = '../Datasets/Data_Set.csv',\n                 blocksize: int = 2, outfilename: str = 'Out.csv')
```

```
    df = dd.read_csv(file_path, blocksize = int(blocksize * 1024 * 1024))
```

```
    with ProgressBar():
```

```
        ChunksSave(df, outfilename)
```

```
    print('Done!')
```

memory intense!

```
out = CSVFileDask2(blocksize = 200)
```

```
[#####] | 100% Completed | 1.67 ss
```

Memory usage for ChunksSave:

Current: 140.01 MB

Peak: 629.59 MB

Done!

Total runtime: 80.73499999998603 seconds



@my_timer

```
def CSVFileDask2(file_path: str = '../Datasets/Data_Set.csv',\n                 blocksize: int = 2, outfilename: str = 'Out.csv')
```

```
    df = dd.read_csv(file_path, blocksize = int(blocksize * 1024 * 1024))
```

```
    with ProgressBar():
```

```
        ChunksSave(df, outfilename)
```

```
    print('Done!')
```

slower, but
requires less
memory

```
out = CSVFileDask2(blocksize = 2)
```

Memory usage for ChunksSave:

Current: 1.56 MB

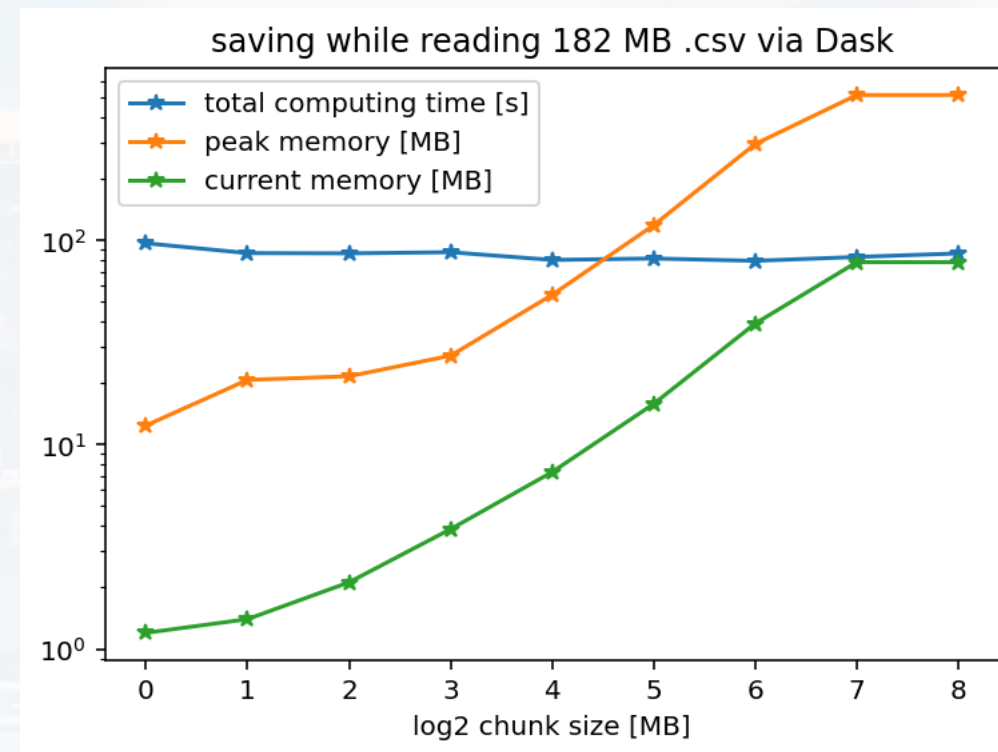
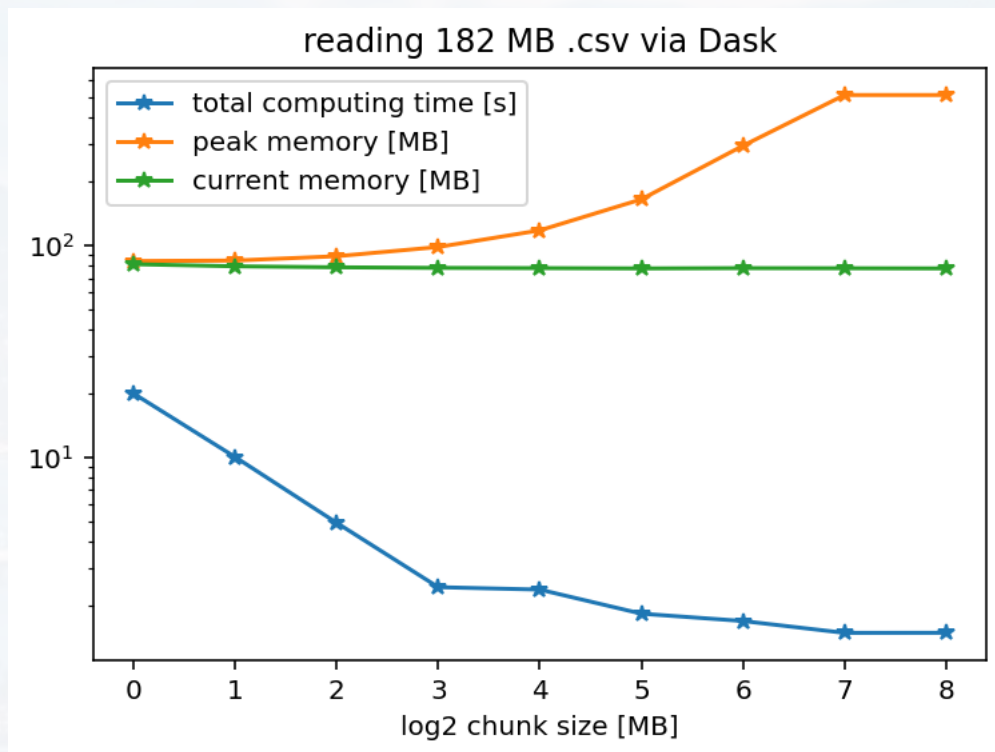
Peak: 20.91 MB

Done!

Total runtime: 102.75 seconds



General tradeoff between memory use and speed!





Outline

Standard Pandas

Line By Line

Chunks With Dask

PySpark



```
Row(_c0=None, _c1='time', _c2='sample 0', _c3='sample 1', _c4='sample 2', _c5='sample 3', _c6='sample 4', _c7='sample 5', _c8='sample 6', _c9='sample 7', _c10='sample 8', _c11='sample 9', _c12='sample 10', _c13='sample 11', _c14='sample 12', _c15='sample 13', _c16='sample 14', _c17='sample 15', _c18='sample 16', _c19='sample 17', _c20='sample 18', _c21='sample 19', _c22='sample 20', _c23='sample 21', _c24='sample 22', _c25='sample 23', _c26='sample 24', _c27='sample 25', _c28='sample 26', _c29='sample 27', _c30='sample 28', _c31='sample 29', _c32='sample 30', _c33='sample 31', _c34='sample 32', _c35='sample 33', _c36='sample 34', _c37='sample 35', _c38='sample 36')
```



summary:

see `LargerThanMemoryExample.ipynb`

reading file line by line using **open**

- **fast**, if only the first few lines
- **no external library** needed
- **low memory** usage
- for **quick overview** of data structure

alternative: PySpark

once we know the file structure:

- **only** reading **data needed** for further analysis
(consistent keywords!)
- read and save data in **chunks**
- save extracted data as **.pkl** (further reading is more efficient)

These steps are usually called preprocessing and are the first step of **Exploratory Data Analysis**



Thank you very much for your attention!

